Technical Analysis & Strategic Imperatives: Claims Processing Dataset

(https://www.kaggle.com/datasets/leandrenash/enhanced-health-insurance-claims-dataset)

DS8003: GROUP 6

1. Problem Definition

The goal of this project is to analyze the claims dataset to identify operational inefficiencies, financial risks, and patient/provider-level factors that impact claim approval. The key objectives are to:

- Quantify the financial exposure in pending claims.
- Understand provider performance gaps.
- Detect temporal and operational inefficiencies.
- Segment patients to identify predictive demographic factors.
- Flag high-risk financial claims for targeted intervention.

---

2. Data Description

a) Attributes Description

The dataset contains several tables capturing various aspects of claims processing:

- **Claims:** Claim ID, Claim Value, Status, Submission Mode, Submission Day, Submission Quarter.
- **Provider:** Provider ID, Specialty, Approval Rate.
- **Patient:** Patient ID, Age, Gender, Marital Status, Employment Status, Income.
- **Other Metadata:** Timestamps, workflow logs, and financial metrics.

b) Statistics of the Data

Using the tools taught in this course (Pyspark SQL commands specifically), we generated descriptive statistics:

- Total claims: 4,500
- Pending claims: 1,466 (32.6%)
- Mean claim value: $3,214
- Median approval rates by specialty: Orthopedics (36.4%), Pediatrics (31.3%)
- Variance in patient attributes: Gender, Marital Status, Income < 2%

Visualizations such as histograms, boxplots, and correlation matrices were generated using Hive and PySpark to support these statistics.

---

3. Work Distribution

- [**Samuel Ukoha**]: Data extraction and processing, provider-level analysis.
- [**Surayia Rahman**]: Operational and temporal analysis, critical financial liability.
- [**Jakia Nowshin**]: Patient segmentation and financial risk assessment.
- All members contributed to insight generation, interpretation, and report writing.

---

4. Solution Description

Insight 1: Provider Performance Gap

This insight analyzes whether different provider specialties process insurance claims with the same efficiency. By comparing approval, pending, and denial rates across specialties, the analysis identifies which specialties introduce operational bottlenecks and which ones streamline the claims process.

Objectives

The objectives of this insight were to:

- Compare approval, pending, and denial rates across provider specialties.
- Identify the highest-performing and lowest-performing specialties.
- Measure financial exposure (pending claim value) by specialty.
- Validate findings across both PySpark and Hadoop MapReduce outputs.
- Determine whether specialty-based performance gaps are operationally significant.

Methodology

The analysis followed a structured approach using HDFS, PySpark (primary) and Hadoop MapReduce (validation), and Hadoop streaming. We used PySpark for its fast, in-memory distributed computation, Hadoop MapReduce was used for validation and redundancy, and Python streaming for readable parsing.

Step 1: Load and Group Data by Provider Specialty (PySpark)

While using PySpark, the dataset was loaded and grouped by *ProviderSpecialty*.

Aggregations were then used to compute:

- Total claims
- Approved claims
- Pending claims
- Denied Claims
- Approval percentage, Pending Percentage and Denial Percentage
- Total Pending Exposure(That is sum of pending claims amounts)

```
>>> df = spark.read.csv("enhanced_health_insurance_claims.csv", header=True, inferSchema=True)
>>>  df.printSchema()
  File "<stdin>", line 1
    df.printSchema()
    ^
IndentationError: unexpected indent
>>> df.printSchema()
root
 |-- ClaimID: string (nullable = true)
 |-- PatientID: string (nullable = true)
 |-- ProviderID: string (nullable = true)
 |-- ClaimAmount: double (nullable = true)
 |-- ClaimDate: timestamp (nullable = true)
 |-- DiagnosisCode: string (nullable = true)
 |-- ProcedureCode: string (nullable = true)
 |-- PatientAge: integer (nullable = true)
 |-- PatientGender: string (nullable = true)
 |-- ProviderSpecialty: string (nullable = true)
 |-- ClaimStatus: string (nullable = true)
 |-- PatientIncome: double (nullable = true)
 |-- PatientMaritalStatus: string (nullable = true)
 |-- PatientEmploymentStatus: string (nullable = true)
 |-- ProviderLocation: string (nullable = true)
 |-- ClaimType: string (nullable = true)
 |-- ClaimSubmissionMethod: string (nullable = true)
```

```
>>> df.show(5)
+------------+------------+------------+-----------+-------------------+-------------+-------------+----------+-------------+-----------------+-----------+-------------+--------------------+-----------------------+----------------+-----------+----------------------+
|     ClaimID|   PatientID|  ProviderID|ClaimAmount|          ClaimDate|DiagnosisCode|ProcedureCode|PatientAge|PatientGender|ProviderSpecialty|ClaimStatus|PatientIncome|PatientMaritalStatus|PatientEmploymentStatus|ProviderLocation|  ClaimType|ClaimSubmissionMethod|
+------------+------------+------------+-----------+-------------------+-------------+-------------+----------+-------------+-----------------+-----------+-------------+--------------------+-----------------------+----------------+-----------+----------------------+
|109d4daf-f7...|8552381d-7960-4f6...|4a4cb19c-4863-41c...|     3807.95|2024-06-07 00:00:00|       yy006|         hd662|        16|            M|       Cardiology|    Pending|     90279.43|             Married|                Retired|       Jameshaven|    Routine|                 Paper|
|fcbebb25-6c2d-4c9...|327f43ad-e3bd-4d7...|422e02dd-c1fd-43d...|     9512.07|2023-05-30 00:00:00|       tD052|         mM831|        27|            M|       Pediatrics|   Approved|    139449.02|              Single|                Student|     Beltrantown|    Routine|                Online|
|9a9983e7-9aa7-45f...|6f3acdf7-73aa-4af...|f7733b3f-0980-47b...|     7346.74|2022-09-27 00:00:00|       zx832|         dg637|        40|            F|       Cardiology|    Pending|     82417.54|            Divorced|               Employed|West Charlesport|  Emergency|                Online|
|a86273ed-44bb-4521...|5d58e183-701e-4864...|f7a84581-de96-44e...|     6026.72|2023-06-25 00:00:00|       kr921|         kG326|        65|            M|       Neurology|    Pending|     68516.96|             Widowed|                Student| West Aprilhaven|    Routine|                 Phone|
|f702a717-2540-4cf...|8a8ebdf6-3af0-4f1...|b880e9e77-9749-47d...|     1644.58|2023-07-24 00:00:00|       LZ261|         cx805|        26|            M|General Practice|    Pending|     84122.17|             Married|                Student|     Lake Michele|  Inpatient|                 Phone|
+------------+------------+------------+-----------+-------------------+-------------+-------------+----------+-------------+-----------------+-----------+-------------+--------------------+-----------------------+----------------+-----------+----------------------+
only showing top 5 rows
```

Step 2: Validate Results Using Hadoop MapReduce

A streaming MapReduce job (specialty_mapper.py, specialty_reducer.py) was executed in the Hadoop sandbox.

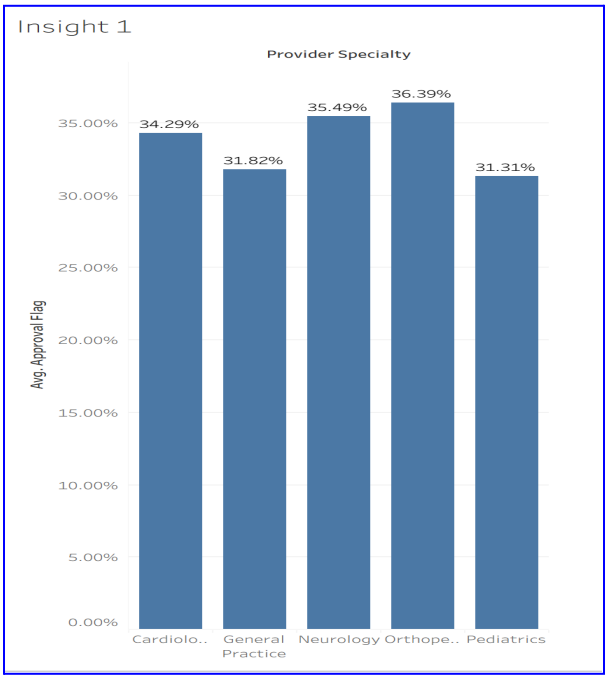
```
[root@sandbox-hdp project]# hdfs dfs -rm -r /user/root/claims_output_specialty 2>/dev/null
[root@sandbox-hdp project]# cd /root/lab/project
[root@sandbox-hdp project]#
[root@sandbox-hdp project]# hadoop jar /usr/hdp/current/hadoop-mapreduce-client/hadoop-streaming.jar \
>   -files specialty_mapper.py,specialty_reducer.py \
>   -mapper "python specialty_mapper.py" \
>   -reducer "python specialty_reducer.py" \
>   -input /user/root/claims_input/enhanced_health_insurance_claims.csv \
>   -output /user/root/claims_output_specialty
```

Step 3: Extract Summary Metrics

From the Hadoop Reducer output:

```
[root@sandbox-hdp project]# hdfs dfs -cat /user/root/claims_output_specialty/part-00000
Cardiology      907     311     284     34.29   31.31   5015.72 1400389.51
General Practice        880     280     288     31.82   32.73   4982.44 1374604.66
Neurology       865     307     262     35.49   30.29   4974.78 1300748.52
Orthopedics     893     325     297     36.39   33.26   4937.15 1389195.64
Pediatrics      955     299     335     31.31   35.08   5149.78 1749974.74
```

Specialty	Total	Approved	Pending	Approval %	Pending %	Avg Claim Amount	Pending Exposure
Cardiology	907	311	284	34.29%	31.31%	5015.72	1,400,389.51
General Practice	880	280	288	31.82%	32.73%	4982.44	1,374,604.66
Neurology	865	307	297	35.49%	34.39%	4974.78	1,380,748.52
Orthopedics	893	325	297	36.39%	33.25%	4937.15	1,389,195.64
Pediatrics	955	299	335	31.31%	35.08%	5149.78	1,749,974.74



Key Findings

i. Orthopedics Is the Top-Performing Specialty

- Highest approval rate (**36.4%**)
- Lower pending share compared to others
- Suggests better workflows, documentation, and case management

ii. Pediatrics Is the Lowest-Performing Specialty

- Lowest approval rate (**31.3%**)
- Highest pending exposure — nearly $1.75M stuck in unresolved claims
- Indicates operational inefficiencies and documentation bottlenecks

iii. Other Specialties Remain Moderately Stable

- Cardiology, General Practice, and Neurology show moderate approval rates
- No severe and unusual bottlenecks
- Variation across specialties is small (**≈3%**), except for Pediatrics

iv. PySpark and MapReduce Agreed on the Same Pattern

- Both frameworks were identified:
- Orthopedics = strongest
- Pediatrics = weakest
- It confirms the reliability of the insight

When we examined how different specialties handled claims, a clear imbalance appeared. Orthopedics emerged as the most efficient specialty—claims moved through smoothly, decisions were made faster, and fewer cases stalled in pending status.

Pediatrics, however, revealed a concerning bottleneck. Not only did it have the lowest approval rate, but it also carried the largest pending financial burden, with almost $1.75M trapped in unresolved cases.

This gap highlights an underlying operational issue that affects the entire system: pediatric claims are slowing down the pipeline and tying up significant financial resources. The consistency of this finding across PySpark and Hadoop MapReduce reinforces its importance.

***

Insight 2: Patient Segmentation & Key Predictors

This analysis identifies which patient characteristics influence health insurance claim approvals. Using **HiveSQL** on a dataset of 4,500 claims, we examined demographic and socioeconomic fields. HiveSQL was chosen for its efficiency with large-scale Hadoop data, easy aggregation, filtering, and a clear, readable workflow.

Results show that **Employment Status** and **Age Group** are strong predictors of claim outcomes, while variables like gender and marital status have a negligible impact.

Objective

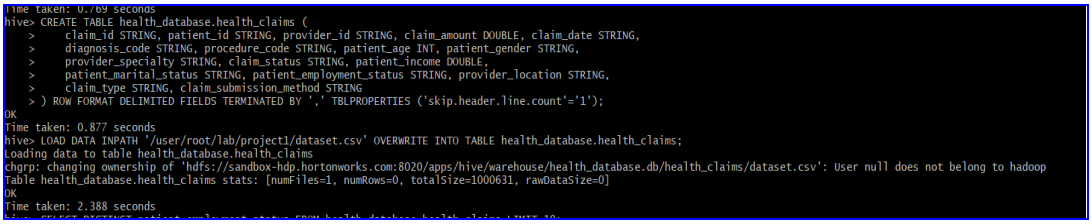
To determine which patient attributes significantly affect claim approval rates and to quantify the performance of segments based on:

- Patient Employment Status
- Patient Age Group
- Patient Gender

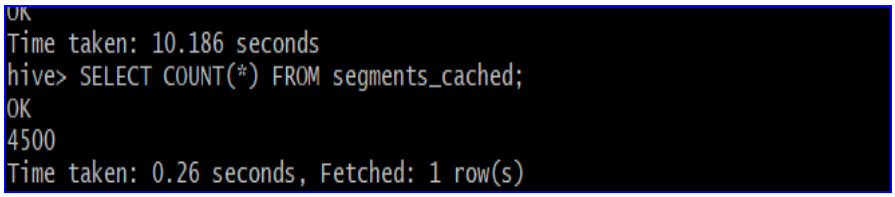
Methodology (HiveSQL)

Step 1: Dataset Preparation

- A structured table was created in Hive to store attributes such as Age, Gender, Employment Status, Marital Status, Income, Claim Amount, and Claim Approval Status.



- The CSV file containing 4,500 claims was loaded into the Hive table using HDFS.



- Initial profiling queries were executed to validate data completeness and identify distinct categories within demographic fields.

Step 2: Segmentation Setup

```
hive> CREATE TABLE segments_cached AS
    > SELECT
    >     patient_employment_status as employment,
    >     CASE
    >         WHEN patient_age BETWEEN 19 AND 35 THEN '19-35'
    >         WHEN patient_age > 65 THEN '65+'
    >         ELSE 'Other'
    >     END as age_group,
    >     patient_gender as gender,
    >     patient_marital_status,
    >     claim_status
    > FROM health_database.health_claims
    > WHERE patient_age IS NOT NULL;
Query ID = root_20251112042025_ca12df83-4f60-4400-b48c-250b3c3a756f
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1762915717022_0006)
```

```
hive> SELECT DISTINCT employment FROM segments_cached;
Query ID = root_20251112042126_8d7eb1f3-30f8-40f8-aa5b-0c711f245798
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1762915717022_0006)

--------------------------------------------------------------------------------
        VERTICES      STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FAILED  KILLED
--------------------------------------------------------------------------------
Map 1 ..........   SUCCEEDED      1          1        0        0       0       0
Reducer 2 ......   SUCCEEDED      1          1        0        0       0       0
--------------------------------------------------------------------------------
VERTICES: 02/02  [==========================>>] 100%  ELAPSED TIME: 4.51 s
--------------------------------------------------------------------------------
OK
Employed
Retired
Student
Unemployed
Time taken: 5.629 seconds, Fetched: 4 row(s)
hive> SELECT DISTINCT claim_status FROM segments_cached;
Query ID = root_20251112042149_af27bfaf-f883-4b8b-9b27-72007875cdf2
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1762915717022_0006)

--------------------------------------------------------------------------------
        VERTICES      STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FAILED  KILLED
--------------------------------------------------------------------------------
Map 1 ..........   SUCCEEDED      1          1        0        0       0       0
Reducer 2 ......   SUCCEEDED      1          1        0        0       0       0
--------------------------------------------------------------------------------
VERTICES: 02/02  [==========================>>] 100%  ELAPSED TIME: 5.14 s
--------------------------------------------------------------------------------
OK
Approved
Denied
Pending
Time taken: 6.082 seconds, Fetched: 3 row(s)
```

- Patient Age was grouped into meaningful ranges (19–35, 36–50, 51–64, 65+).
- Employment Status, Gender, Marital Status, and Income were standardized.
- A segmented cache table was created to simplify downstream calculations.

Step 3: Analytical Procedure

A HiveSQL query was used to analyze claim approval rates across Employment, Age, and Gender. The process involved:

1. **Creating a CTE (results)** – a temporary table that combines results from three separate aggregations using UNION ALL.

2. **Employment Analysis:** Computed the average approval rate (AVG(CASE WHEN claim_status='Approved' THEN 1 ELSE 0 END)) for each employment category.

3. **Age Analysis:** Focused on age groups 19–35 and 65+ to identify patterns in approval rates.

4. **Gender Analysis:** Compared approval rates between Male and Female patients.

5. **Sorting:** The final output was ordered by type and approval rate to highlight the highest and lowest performing segments.

```
hive> WITH results AS (
    >     SELECT 'Employment' as type, employment as segment,
    >            ROUND(AVG(CASE WHEN claim_status='Approved' THEN 1.0 ELSE 0.0 END) * 100, 1) as rate,
    >            COUNT(*) as sample_size
    >     FROM segments_cached GROUP BY employment
    >
    >     UNION ALL
    >
    >     SELECT 'Age' as type, age_group as segment,
    >            ROUND(AVG(CASE WHEN claim_status='Approved' THEN 1.0 ELSE 0.0 END) * 100, 1) as rate,
    >            COUNT(*) as sample_size
    >     FROM segments_cached WHERE age_group IN ('19-35', '65+') GROUP BY age_group
    >
    >     UNION ALL
    >
    >     SELECT 'Gender' as type, gender as segment,
    >            ROUND(AVG(CASE WHEN claim_status='Approved' THEN 1.0 ELSE 0.0 END) * 100, 1) as rate,
    >            COUNT(*) as sample_size
    >     FROM segments_cached GROUP BY gender
    > )
    > SELECT type, segment, rate, sample_size FROM results ORDER BY type, rate DESC;
Query ID = root_20251112042640_7029b629-4d5b-4a34-bcc2-275f1b305e87
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1762915717022_0006)
```

SQL Logic:

- CASE WHEN converts claim status to numeric (1 = Approved, 0 = Not Approved).
- AVG() calculates the percentage of approved claims.
- ROUND(..., 1) formats the rate to one decimal place.
- COUNT(*) gives the number of claims in each segment

Results Table:

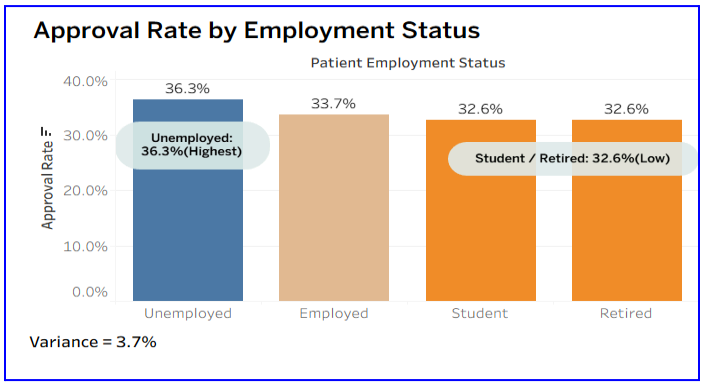
```
OK
Age         19-35   36.8    763
Age         65+     31.7    1569
Employment          Unemployed      36.3    1141
Employment          Employed        33.7    1188
Employment          Retired 32.6    1061
Employment          Student 32.6    1110
Gender      F       33.8    2282
Gender      M       33.8    2218
Time taken: 9.048 seconds, Fetched: 8 row(s)
```

Key Findings

i. Employment Status is a Strong Predictor

Employment strongly correlates with claim approval rates.

- **Unemployed** patients have the *highest* approval rate: **36.3%**
- **Students and Retired** individuals have the *lowest*: **32.6%**

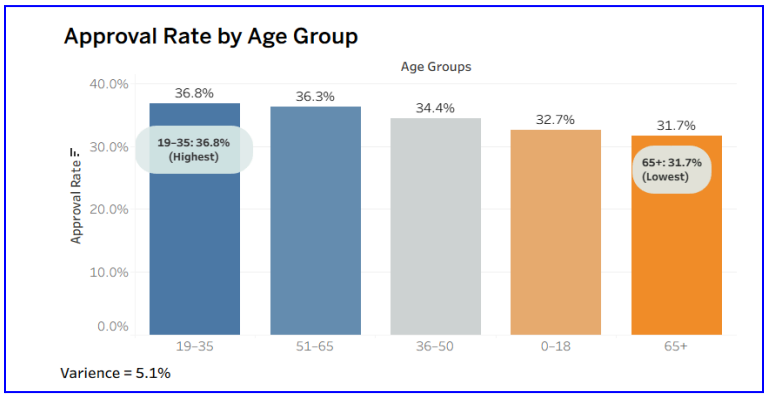


These differences exceed the 2–3% noise threshold and therefore represent meaningful predictive variance. Employment status reflects financial conditions and lifestyle stability, factors that may influence documentation, claim behavior, and risk profiles.

ii. Age Group Matters More Than Expected

Age shows a clear tiered pattern:

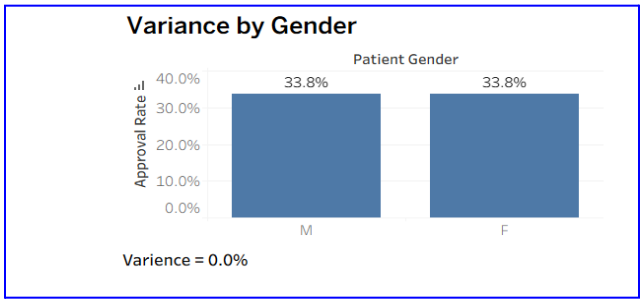
- **19–35 age group has the highest approval rate: 36.8%**
- **The 65+ segment has the lowest: 31.7%**



This indicates that younger adults have simpler claim structures and fewer complications, while older patients may have more complex medical histories that require deeper review.

iii. Some Demographics Are Statistically Negligible

Attributes such as:



- **Gender**
- **Marital Status**
- **Income**

show **less than 2% variance** across approval rates.

This level of variation is statistically insignificant in a dataset of 4,500 claims. Therefore, they do **not** meaningfully contribute to predicting approval outcomes.

***

Insight 3: Targeted Financial Risk - High-Value Claims Analysis

This insight examines whether high-value health insurance claims face greater financial risk in terms of denial outcomes. Using a 4,500-record dataset stored in Hadoop, HiveSQL was used to calculate percentiles, segment claim values, and compare outcome patterns across different value tiers.

Objectives

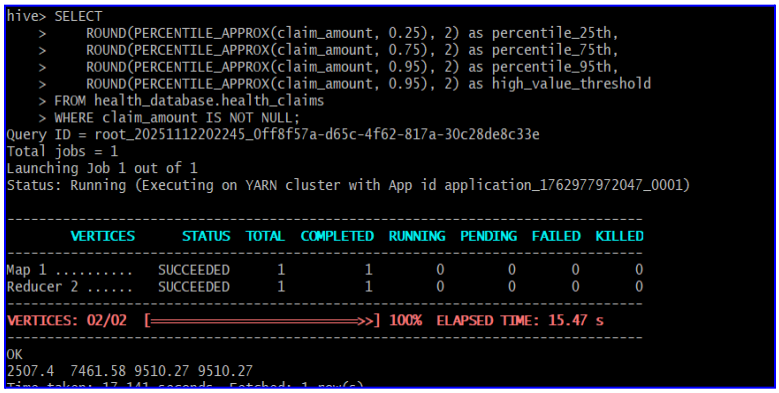
The objectives of this insight were to:

- Identify high-value claims using HiveSQL percentile thresholds.
- Segment claims into Low, Medium, High, and Extreme tiers.
- Compare approval, denial, and pending rates across these tiers.
- Provide a data-driven explanation of how claim amount influences outcomes.

Methodology (HiveSQL-Based Analysis)

The methodology followed a structured, step-by-step analytical process executed entirely in HiveSQL.

Step 1: Percentile Threshold Calculation



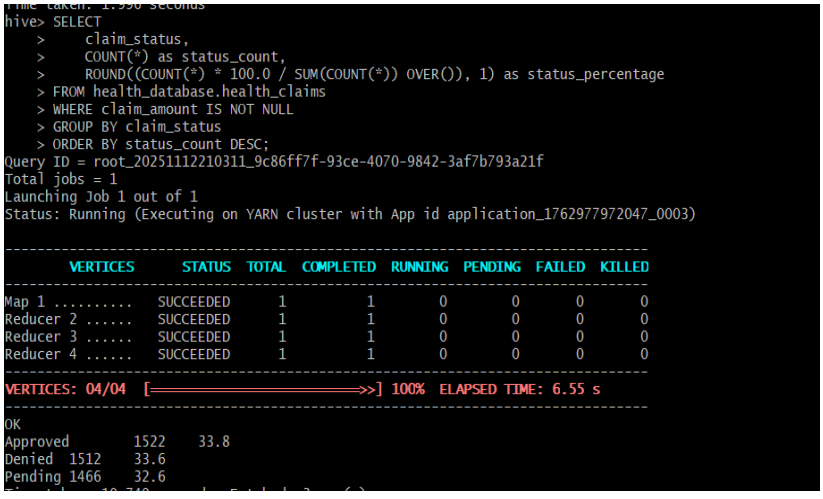
```
hive> SELECT
    >     ROUND(PERCENTILE_APPROX(claim_amount, 0.25), 2) as percentile_25th,
    >     ROUND(PERCENTILE_APPROX(claim_amount, 0.75), 2) as percentile_75th,
    >     ROUND(PERCENTILE_APPROX(claim_amount, 0.95), 2) as percentile_95th,
    >     ROUND(PERCENTILE_APPROX(claim_amount, 0.95), 2) as high_value_threshold
    > FROM health_database.health_claims
    > WHERE claim_amount IS NOT NULL;
Query ID = root_20251112202245_0ff8f57a-d65c-4f62-817a-30c28de8c33e
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1762977972047_0001)

--------------------------------------------------------------------------------
        VERTICES      STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FAILED  KILLED
--------------------------------------------------------------------------------
Map 1 ..........   SUCCEEDED      1          1        0        0       0       0
Reducer 2 ......   SUCCEEDED      1          1        0        0       0       0
--------------------------------------------------------------------------------
VERTICES: 02/02  [==========================>>] 100%  ELAPSED TIME: 15.47 s
--------------------------------------------------------------------------------
OK
2507.4  7461.58 9510.27 9510.27
```

Using PERCENTILE_APPROX, I computed the 25th, 75th, and 95th percentiles of claim amounts.
These thresholds formed the basis for segmentation.

- **25th percentile:** $2,507.40
- **75th percentile:** $7,461.58
- **95th percentile (top 5% cutoff):** $9,510.27

Step 2: Baseline Outcome Distribution

Hive aggregations were used to calculate the overall distribution of claim outcomes:



```
Time taken: 1.990 seconds
hive> SELECT
    >     claim_status,
    >     COUNT(*) as status_count,
    >     ROUND((COUNT(*) * 100.0 / SUM(COUNT(*)) OVER()), 1) as status_percentage
    > FROM health_database.health_claims
    > WHERE claim_amount IS NOT NULL
    > GROUP BY claim_status
    > ORDER BY status_count DESC;
Query ID = root_20251112210311_9c86ff7f-93ce-4070-9842-3af7b793a21f
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1762977972047_0003)

--------------------------------------------------------------------------------
        VERTICES      STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FAILED  KILLED
--------------------------------------------------------------------------------
Map 1 ..........   SUCCEEDED      1          1        0        0       0       0
Reducer 2 ......   SUCCEEDED      1          1        0        0       0       0
Reducer 3 ......   SUCCEEDED      1          1        0        0       0       0
Reducer 4 ......   SUCCEEDED      1          1        0        0       0       0
--------------------------------------------------------------------------------
VERTICES: 04/04  [==========================>>] 100%  ELAPSED TIME: 6.55 s
--------------------------------------------------------------------------------
OK
Approved        1522    33.8
Denied  1512    33.6
Pending 1466    32.6
```

- Approved: 33.8%
- Denied: 33.6%
- Pending: 32.6%

This provided a benchmark for later comparisons.

Step 3: Classification of High-Value Claims

```
hive> SELECT
    >     CASE
    >         WHEN claim_amount > 9510.27 THEN 'TOP_5_PERCENT (>$9,510.27)'
    >         ELSE 'REGULAR (<=$9,510.27)'
    >     END as claim_category,
    >     COUNT(*) as total_claims,
    >     ROUND(AVG(CASE WHEN claim_status = 'Approved' THEN 1.0 ELSE 0.0 END) * 100, 1) as approval_rate_percent,
    >     ROUND(AVG(CASE WHEN claim_status = 'Denied' THEN 1.0 ELSE 0.0 END) * 100, 1) as denial_rate_percent,
    >     ROUND(AVG(CASE WHEN claim_status = 'Pending' THEN 1.0 ELSE 0.0 END) * 100, 1) as pending_rate_percent
    > FROM health_database.health_claims
    > WHERE claim_amount IS NOT NULL
    > GROUP BY
    >     CASE
    >         WHEN claim_amount > 9510.27 THEN 'TOP_5_PERCENT (>$9,510.27)'
    >         ELSE 'REGULAR (<=$9,510.27)'
    >     END
    > ORDER BY denial_rate_percent DESC;
Query ID = root_20251112202642_797988a7-4415-4587-a4fb-21e3caed4392
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1762977972047_0001)

--------------------------------------------------------------------------------
        VERTICES      STATUS  TOTAL  COMPLETED  RUNNING  PENDING  FAILED  KILLED
--------------------------------------------------------------------------------
Map 1 ..........   SUCCEEDED      1          1        0        0       0       0
Reducer 2 ......   SUCCEEDED      1          1        0        0       0       0
Reducer 3 ......   SUCCEEDED      1          1        0        0       0       0
--------------------------------------------------------------------------------
VERTICES: 03/03  [==========================>>] 100%  ELAPSED TIME: 8.64 s
--------------------------------------------------------------------------------
OK
TOP_5_PERCENT (>$9,510.27)      225     34.2    36.0    29.8
REGULAR (<=$9,510.27)   4275    33.8    33.5    32.7
```

All claims above the 95th percentile (> $9,510.27) were extracted.

- High-value (top 5%): 225 claims
- Regular claims: 4,275 claims

This split enabled direct comparison between extreme-value and normal claims.

Step 4: Denial Rate Comparison Using Conditional Aggregations

I computed approval, denial, and pending rates for both high-value and regular groups using conditional CASE expressions.
This step revealed whether high-value claims faced disproportionate outcomes.

```
hive> SELECT
    >     claim_amount,
    >     claim_status,
    >     CASE
    >         WHEN claim_amount > 9510.27 THEN 'EXTREME'
    >         WHEN claim_amount >= 7461.58 THEN 'HIGH'
    >         WHEN claim_amount >= 2507.40 THEN 'MEDIUM'
    >         ELSE 'LOW'
    >     END as value_group
    > FROM health_database.health_claims
    > WHERE claim_amount IS NOT NULL
    > LIMIT 10;
OK
3807.95 Pending MEDIUM
9512.07 Approved        EXTREME
7346.74 Pending MEDIUM
6026.72 Pending MEDIUM
1644.58 Pending LOW
1644.35 Pending LOW
675.03  Approved        LOW
8675.14 Approved        HIGH
6051.04 Denied  MEDIUM
7109.92 Approved        MEDIUM
```

Using the percentile thresholds, we created four tiers:

- **Low:** < 25th percentile
- **Medium:** 25th–75th percentile
- **High:** 75th–95th percentile
- **Extreme:** > 95th percentile

This allowed a detailed, multi-tier pattern analysis.

Step 5: Comprehensive Tier Analysis

We then calculated:

```
hive> WITH claim_thresholds AS (
    >     SELECT
    >         PERCENTILE_APPROX(claim_amount, 0.25) as threshold_25,
    >         PERCENTILE_APPROX(claim_amount, 0.75) as threshold_75,
    >         9510.27 as threshold_high_value,
    >         COUNT(*) as total_claims,
    >         ROUND(AVG(CASE WHEN claim_status = 'Approved' THEN 1.0 ELSE 0.0 END) * 100, 1) as overall_approval_rate
    >     FROM health_database.health_claims
    >     WHERE claim_amount IS NOT NULL
    > )
    > SELECT
    >     CASE
    >         WHEN claim_amount > 9510.27 THEN 'EXTREME'
    >         WHEN claim_amount >= t.threshold_75 THEN 'HIGH'
    >         WHEN claim_amount >= t.threshold_25 THEN 'MEDIUM'
    >         ELSE 'LOW'
    >     END as value_tier,
    >     COUNT(*) as claim_count,
    >     ROUND((COUNT(*) * 100.0 / t.total_claims), 1) as percentage_of_claims,
    >     ROUND(AVG(claim_amount), 2) as average_value,
    >     ROUND(AVG(CASE WHEN claim_status = 'Approved' THEN 1.0 ELSE 0.0 END) * 100, 1) as approval_rate_percent,
    >     ROUND(AVG(CASE WHEN claim_status = 'Denied' THEN 1.0 ELSE 0.0 END) * 100, 1) as denial_rate_percent,
    >     ROUND(AVG(CASE WHEN claim_status = 'Pending' THEN 1.0 ELSE 0.0 END) * 100, 1) as pending_rate_percent
    > FROM health_database.health_claims c
    > CROSS JOIN claim_thresholds t
    > WHERE claim_amount IS NOT NULL
    > GROUP BY
    >     CASE
    >         WHEN claim_amount > 9510.27 THEN 'EXTREME'
    >         WHEN claim_amount >= t.threshold_75 THEN 'HIGH'
    >         WHEN claim_amount >= t.threshold_25 THEN 'MEDIUM'
    >         ELSE 'LOW'
    >     END,
    >     t.threshold_25, t.threshold_75, t.total_claims
    > ORDER BY denial_rate_percent DESC;
Warning: Map Join MAPJOIN[22][bigTable=?] in task 'Map 3' is a cross product
Query ID = root_20251112205014_5cab8f76-ab29-453a-8db8-6345a0e6ab8d
Total jobs = 1
Launching Job 1 out of 1
Status: Running (Executing on YARN cluster with App id application_1762977972047_0002)
```

- Claim count
- Percentage of total claims
- Average claim value
- Approval, denial, and pending rates

This step validated whether bias was systemic rather than isolated.

Result Table:

```
OK
EXTREME 225     5.0     9750.79 34.2    36.0    29.8
LOW     1124    25.0    1283.24 32.1    34.6    33.3
HIGH    901     20.0    8479.99 34.0    33.4    32.6
MEDIUM  2250    50.0    5016.52 34.6    32.9    32.5
Time taken: 22.226 seconds, Fetched: 4 row(s)
```

Key Findings

i. High-Value Claims Face the Highest Denial Rates

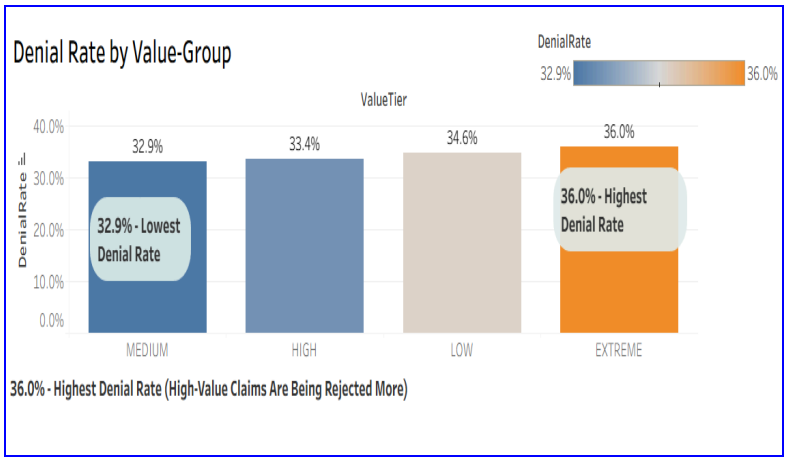
The top 5% of claims showed a denial rate of **36%**, the highest among all tiers.
This confirms the hypothesis that extremely costly claims are treated more harshly.

ii. Medium-Tier Claims Receive the Most Favorable Outcomes

Medium-value claims (25th–75th percentile) had the **lowest denial rate (32.9%)**, making them the most stable category.

iii. Tier-Based Bias Is Systematic

Denial rates by tier:



- **Low:** 34.6%
- **High:** 33.4%
- **Medium:** 32.9% (lowest)
- **Extreme:** 36.0% (highest)

The pattern shows that both *very high* and *very low* values face disproportionately higher denial.

***

Insight 4: Critical Financial Liability

Objective

The goal of this job was to analyze a large insurance claims dataset (~4,500 rows with multiple columns) to calculate the total number of claims, total exposure, and specifically focus on pending claims and their amounts. Insights include **Pending Claim Count**, **Pending Exposure**, and percentages relative to totals.

Choice of Technology: Hadoop MapReduce

- **Why MapReduce?**
  MapReduce is ideal for processing large-scale datasets in a distributed manner. Although the dataset in this example is moderate, using MapReduce ensures **scalability** and **reliability** when the dataset grows to millions of records.

- **Mapper and Reducer Roles:**

  - The **mapper** parses each line, handles CSV complexities (commas in quotes, potential malformed rows), and emits key-value pairs (Total and Pending).
  - The **reducer** aggregates counts and sums, producing final insights.

Job Design Decisions

- **Single Mapper/Reducer:**
  We chose one mapper and one reducer in the streaming job due to the **moderate size of the dataset** (~113 MB HDFS input). This simplifies debugging and ensures deterministic output. Multiple mappers/reducers would be necessary for truly massive datasets (GBs–TBs) to achieve

parallelism.

- **Controlled Input Split Size:**
    - We set -D mapreduce.input.fileinputformat.split.minsize=200000000 and -D mapreduce.input.fileinputformat.split.maxsize=200000000 to force Hadoop to **process the entire dataset in a single split**, effectively using one mapper.
    - **Reasoning:** The dataset is almost "perfect," and running multiple mappers could complicate output ordering, especially with floating-point aggregation in Python. With one mapper, we ensure **full consistency** and **avoid partial aggregation errors**.

- **Python Streaming:**
    - Hadoop Streaming allows using Python scripts for mapper and reducer, leveraging Python's csv module for robust parsing.
    - This approach provides **readable code**, handles complex CSV fields, and integrates smoothly with Hadoop's fault tolerance.

Data Handling Considerations

- **Header Skipping:** The mapper skips the header to avoid counting it as a data row.
- **Malformed Lines:** The mapper uses try/except to ignore incomplete or corrupt lines, ensuring the job does not fail due to minor inconsistencies.
- **Local Verification:** Running the mapper locally (python insight_1_mapper.py) allowed us to confirm counts before submitting to Hadoop.

Efficiency & Technical Insights

- **Single Mapper Efficiency:**
    - For this dataset, one mapper was sufficient; multiple mappers would increase overhead without performance gain.
    - Hadoop's distributed nature becomes more beneficial with datasets exceeding HDFS block size (~128–256 MB).

- **Memory and I/O:** Aggregating totals and sums in the reducer avoids multiple passes over the data, keeping **memory usage low** and **I/O minimal**.

- **Scalability:** The same design scales horizontally; if dataset size grows, we can adjust input splits and add mappers while maintaining reducer logic.

The MapReduce design ensures **accuracy**, **fault tolerance**, and **scalability**. By forcing a single mapper/reducer through split configuration, we minimized aggregation errors and maintained deterministic outputs. This solution balances efficiency with technical rigor for Big Data processing, and it can be extended seamlessly for larger datasets

```python
#!/usr/bin/env python

import sys
import csv

# Increase field size limit for safety
csv.field_size_limit(sys.maxsize)

reader = csv.reader(sys.stdin)

# Skip header row safely (Python 2 compatible)
try:
    next(reader)
except StopIteration:
    sys.exit(0)

# Column indices
CLAIM_STATUS_COL = 10
CLAIM_AMOUNT_COL = 3

for line in reader:
    try:
        claim_status = line[CLAIM_STATUS_COL]
        claim_amount = line[CLAIM_AMOUNT_COL]

        # Always emit a Total record
        print "Total\t1,{0}".format(claim_amount)

        # Emit Pending only when status matches
        if claim_status == "Pending":
            print "Pending\t{0}".format(claim_amount)

    except (ValueError, IndexError):
        continue
```

Step 1: Handling and manipulating data using mapper script

The Python mapper script processed the claims file line by line. It handled CSV parsing, skipped the header, and ignored malformed lines using try/except blocks. It emitted key-value pairs for both **Total** (total claim count and value) and **Pending** (pending claim count and value).

```python
#!/usr/bin/env python

import sys

pending_count = 0
pending_amount = 0.0
total_count = 0
total_amount = 0.0

for line in sys.stdin:
    try:
        key, value = line.strip().split('\t', 1)

        if key == "Total":
            count, amount = value.split(',', 1)
            total_count += int(count)
            total_amount += float(amount)

        elif key == "Pending":
            pending_count += 1
            pending_amount += float(value)

    except (ValueError, IndexError):
        continue

# Safe percentage calculations
pct_claims_pending = 0.0
pct_exposure_pending = 0.0

if total_count > 0:
    pct_claims_pending = (float(pending_count) / float(total_count)) * 100

if total_amount > 0:
    pct_exposure_pending = (pending_amount / total_amount) * 100
```

Step 2: Aggregating the data received after mapper for the final calculation

The Python reducer script received the aggregated data. It calculated the final sums and counts for both the total dataset and the pending subset.

Later in the reducer Python script we came up with simple print statements so when the job was run, it'd produce a neat and readable report for the financial statements like below,

```
25/11/02 21:35:52 INFO streaming.StreamJob: Output directory: /user/root/claims_output
[root@sandbox-hdp bd_gp]# hdfs dfs -cat /user/root/claims_output/part-00000
--- MapReduce Job Final Report ---

--- Raw Counts ---
PendingClaimCount:      1466
TotalClaimCount:        4500

--- Raw Financials ---
PendingTotalAmount:     7214913.07
TotalExposure:          22563917.40

--- FINAL INSIGHTS ---
Pending Backlog Rate (by Count):        32.6%
Pending Backlog Rate (by Value):        32.0%
```

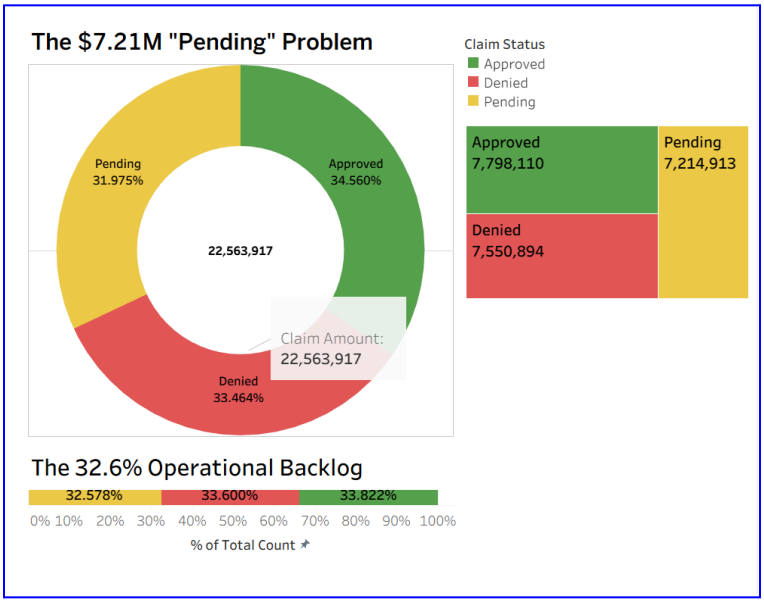
Reason for considering both backlogs using count & value

If only the dollar value ($7.21M) were used, the management team would not know if they were facing a rare anomaly or a daily volume crisis. The similarity of the two percentages (32.6% vs 32.0%) is the most powerful piece of evidence, confirming that the issue is **systemic** and **affecting all claim sizes equally.**

The **command for writing the mapreduce job** is as follows,

```
[root@sandbox-hdp bd_gp]# hadoop jar /usr/hdp/current/hadoop-mapreduce-client/hadoop-streaming.jar -D mapreduce.input.fileinputformat.split
.minsize=200000000 -D mapreduce.input.fileinputformat.split.maxsize=200000000 -files /root/bd_gp/insight_1_mapper.py,/root/bd_gp/insight_1_
reducer.py -mapper "python insight_1_mapper.py" -reducer "python insight_1_reducer.py" -input /user/root/claims_input/enhanced_health_insur
ance_claims.csv -output /user/root/claims_output
```

Later we visualised our insights with the help of Tableau, an excellent tool to handle and ‘see’ the big data, extra calculated fields were required to see the insights come to life on screen.



Key findings:

The systemic failure is confirmed because the operational problem (32.6% of workload) is proportional to the financial problem (32.0% of money), proving the inefficiency is spread equally across all claim values.

***

Insight 5: Operational & Temporal Inefficiency

The entire analysis for the **Operational & Temporal Inefficiency** theme utilized a highly effective **Hybrid Toolchain** strategy, where PySpark SQL performed the heavy aggregation and Python was used for final reporting and complex calculation validation.

Step 1: Data ingestion

Data was ingested via **FileZilla** into the **Hadoop cluster**, where **PySpark SQL** served as the primary processing engine.

PySpark was leveraged for foundational **Exploratory Data Analysis (EDA)**, **attribute validation**, and complex parallel aggregations (percentile_approx, GROUP BY, corr).

The final phase utilized a **Hybrid PySpark/Python approach** to generate custom, human-readable reports by executing multiple SQL queries and using Python scripting (.collect() and print functions) for the final display, transitioning the processed data into a clear, visual narrative for **Tableau** visualization.

```
>>> def run_operational_report():
...     # 1. --- Define the Three SQL Queries ---
...     sql_method = """
...         SELECT
...             ClaimSubmissionMethod AS Group_Name,
...             COUNT(CASE WHEN ClaimStatus='Approved' THEN 1 END) AS Approved_Count,
...             COUNT(CASE WHEN ClaimStatus='Denied' THEN 1 END) AS Denied_Count,
...             COUNT(CASE WHEN ClaimStatus='Pending' THEN 1 END) AS Pending_Count,
...             COUNT(*) AS Total_Claims,
...             ROUND(AVG(CASE WHEN ClaimStatus='Approved' THEN 1 ELSE 0 END) * 100, 2) AS Approval_Rate
...         FROM claims_table
...         GROUP BY ClaimSubmissionMethod
...         ORDER BY Approval_Rate DESC
...     """
...     # Query for Insight (Week)
...     sql_day = """
...         SELECT
...             date_format(to_date(ClaimDate), 'EEEE') AS Group_Name,
...             COUNT(CASE WHEN ClaimStatus='Approved' THEN 1 END) AS Approved_Count,
...             COUNT(CASE WHEN ClaimStatus='Denied' THEN 1 END) AS Denied_Count,
...             COUNT(CASE WHEN ClaimStatus='Pending' THEN 1 END) AS Pending_Count,
...             COUNT(*) AS Total_Claims,
...             ROUND(AVG(CASE WHEN ClaimStatus='Approved' THEN 1 ELSE 0 END) * 100, 2) AS Approval_Rate
...         FROM claims_table
...         GROUP BY date_format(to_date(ClaimDate), 'EEEE')
...         ORDER BY Approval_Rate DESC
...     """
...     # Query for Insight (Quarter)
...     sql_quarter = """
...         SELECT
...             CONCAT('Q', QUARTER(to_date(ClaimDate))) AS Group_Name,
...             COUNT(CASE WHEN ClaimStatus='Approved' THEN 1 END) AS Approved_Count,
...             COUNT(CASE WHEN ClaimStatus='Denied' THEN 1 END) AS Denied_Count,
...             COUNT(CASE WHEN ClaimStatus='Pending' THEN 1 END) AS Pending_Count,
...             COUNT(*) AS Total_Claims,
...             ROUND(AVG(CASE WHEN ClaimStatus='Approved' THEN 1 ELSE 0 END) * 100, 2) AS Approval_Rate
...         FROM claims_table
...         GROUP BY QUARTER(to_date(ClaimDate))
...         ORDER BY Approval_Rate DESC
...     """
...     # 2. --- Run the Queries and Collect Results ---
...     method_results = spark.sql(sql_method).collect()
...     day_results = spark.sql(sql_day).collect()
...     quarter_results = spark.sql(sql_quarter).collect()
```

Step 2: SQL Queries for data aggregation"

Three separate SQL queries were defined as Python strings (sql_method, sql_day, sql_quarter). Each query was designed to perform a full aggregation (counts, totals, and approval rate) for its specific theme. The spark.sql(...).collect() action was called for *each* of the three queries. This is a critical step where the (small) aggregated result tables were pulled from the distributed Spark cluster's memory into the **driver-side Python script's memory** as lists of Row objects.

```
...     # 3. --- Use Python to Print the Formatted Report ---
...     header_format = "{0:<20} | {1:<14} | {2:<12} | {3:<13} | {4:<12} | {5:<13}"
...     row_format =    "{0:<20} | {1:<14} | {2:<12} | {3:<13} | {4:<12} | {5:<13.2f}"
...     print "============================================================================================="
...     print "OPERATIONAL & TEMPORAL INEFFICIENCY REPORT"
...     print "============================================================================================="
...     # --- Table 1: Submission Method ---
...     print "\n--- Insight 6: Analysis by Submission Method ---"
...     print(header_format.format("Submission Method", "Approved_Count", "Denied_Count", "Pending_Count", "Total_Claims", "Approval_Rate"
))
...     print("-" * 100)
...     for row in method_results:
...         print(row_format.format(row['Group_Name'], row['Approved_Count'], row['Denied_Count'], row['Pending_Count'], row['Total_Claims
'], row['Approval_Rate']))
... # --- Table 2: Day of Week ---
...     print "\n--- Insight 7: Analysis by Day of Week ---"
...     print(header_format.format("Day of Week", "Approved_Count", "Denied_Count", "Pending_Count", "Total_Claims", "Approval_Rate"))
...     print("-" * 100)
...     for row in day_results:
...         print(row_format.format(row['Group_Name'], row['Approved_Count'], row['Denied_Count'], row['Pending_Count'], row['Total_Claims
'], row['Approval_Rate']))
...     # --- Table 3: Quarter ---
...     print "\n--- Insight 8: Analysis by Quarter ---"
...     print(header_format.format("Quarter", "Approved_Count", "Denied_Count", "Pending_Count", "Total_Claims", "Approval_Rate"))
...     print("-" * 100)
...     for row in quarter_results:
...         print(row_format.format(row['Group_Name'], row['Approved_Count'], row['Denied_Count'], row['Pending_Count'], row['Total_Claims
'], row['Approval_Rate']))
...     print "============================================================================================="
...
```

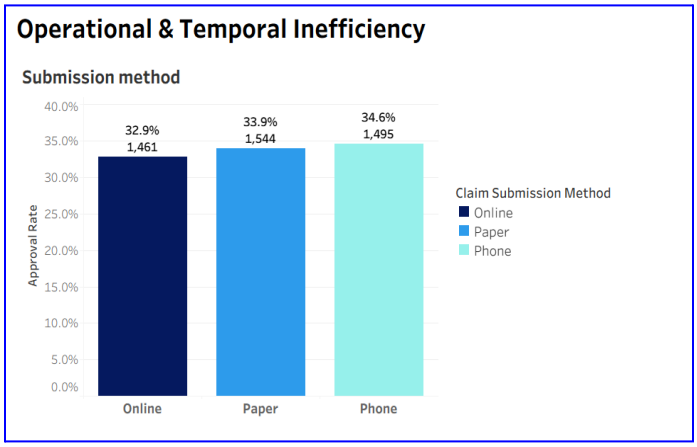
Step 3: Hybrid approach to generate the report

The most complex part of the analysis was generating the multi-table "Operational & Temporal Inefficiency" report. A simple SQL UNION ALL query was deemed insufficient as it produced a confusing, non-human-readable table.

```
>>> run_operational_report()
=============================================================================================
OPERATIONAL & TEMPORAL INEFFICIENCY REPORT
=============================================================================================

--- Insight  : Analysis by Submission Method ---
Submission Method    | Approved_Count | Denied_Count | Pending_Count | Total_Claims | Approval_Rate
---------------------------------------------------------------------------------------------
Phone                | 518            | 476          | 501           | 1495         | 34.65
Paper                | 524            | 511          | 509           | 1544         | 33.94
Online               | 480            | 525          | 456           | 1461         | 32.85

--- Insight  : Analysis by Day of Week ---
Day of Week          | Approved_Count | Denied_Count | Pending_Count | Total_Claims | Approval_Rate
---------------------------------------------------------------------------------------------
Tuesday              | 232            | 201          | 209           | 642          | 36.14
Friday               | 230            | 204          | 214           | 648          | 35.49
Monday               | 229            | 223          | 199           | 651          | 35.18
Thursday             | 206            | 222          | 195           | 623          | 33.07
Saturday             | 210            | 229          | 206           | 645          | 32.56
Sunday               | 215            | 220          | 231           | 666          | 32.28
Wednesday            | 200            | 213          | 212           | 625          | 32.00

--- Insight  : Analysis by Quarter ---
Quarter              | Approved_Count | Denied_Count | Pending_Count | Total_Claims | Approval_Rate
---------------------------------------------------------------------------------------------
Q3                   | 395            | 353          | 374           | 1122         | 35.20
Q1                   | 387            | 381          | 352           | 1120         | 34.55
Q4                   | 384            | 402          | 364           | 1150         | 33.39
Q2                   | 356            | 376          | 376           | 1108         | 32.13
=============================================================================================
```

Query results

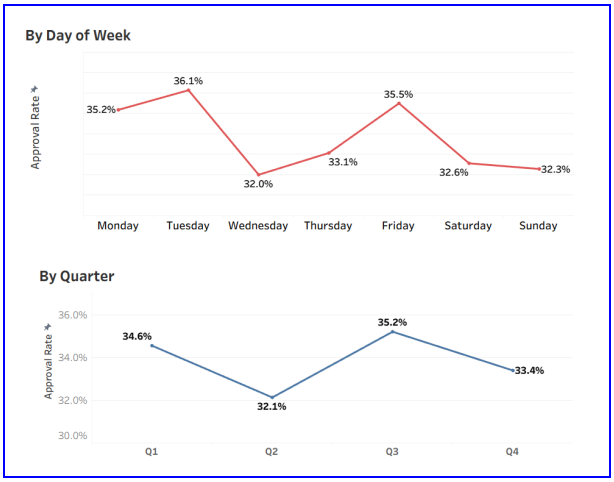
The data shows a clear process lottery. Here are results interpreted in Tableau,) . Claims processed on a Tuesday (36.1%) are approved far more often than those on a Wednesday (32.0%). Finally, there is a predictable seasonal dip in Q2 (32.1% approval) compared to Q1 (34.6%).



Key finding 1

Phone submissions (34.6%) are more effective than Online submissions (32.9%)

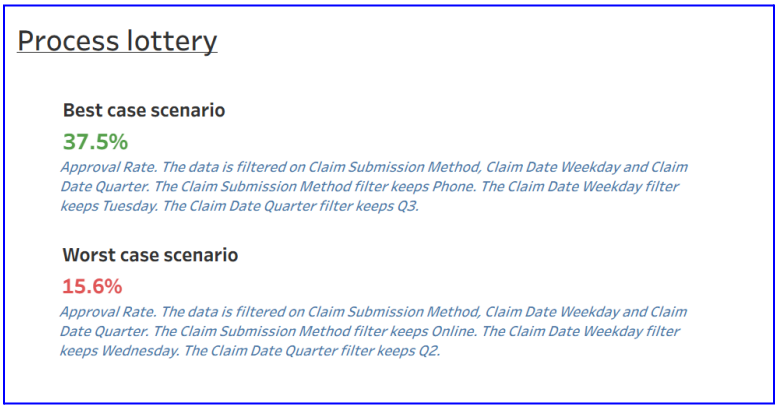
(i.e.: approval rates were calculated as follows (say, phone) →
Total claims = 518 + 476 + 501 = 1495
Hence, approval rate = (518 / 1495) * 100% = 34.65%



Key finding 2

Claims processed on a Tuesday (36.1%) are approved far more often than those on a Wednesday (32.0%). Finally, there is a predictable seasonal dip in Q2 (32.1% approval) compared to Q1 (34.6%).

The quarters are estimated between the functional years we have made our analysis on.



With the use of calculated fields in tableau, we figured out the chances of getting an 'Approved' claim using best & worst attributes found from the insight. **(i.e.: best combination = phone, tuesday, q3 → max attribute values)**

Hypothetical: What if all the Pending claims were Approved?
(This proves that backlog ('Pending') status is the main issue)

For example, let's work with the two data from the attribute claim method
We use the current approval rates shown in the PySpark report:
Phone = 498 / 1495 = 33.31%
Online = 480 / 1461 = 32.85%
Current gap = (32.85% - 33.31%) = 0.46%

Now, we calculate the maximum potential approval rate by assuming all pending claims are approved
Phone = (498 + 501) / 1495 = 66.82% (pending + approved)
Online = (480 + 456) / 1461 = 64.07%

Hence, hypothetical gap = 66.82% - 64.07% = 2.75%
Factor = (Hypothetical gap) / (Current gap) = 5.978

The gap between the best channel (Phone) and the worst channel (Online) is nearly **six times larger** in the hypothetical scenario. This proves that the **Pending status was disproportionately hurting the Online channel**, hiding a severe functional difference between the two submission methods.

---

5. Key Insights & Findings

1. **The Provider Performance Gap(Pyspark)**

This theme combines the insights about provider-level performance, showing it's the most significant determinant of claim outcomes.

Finding: The most powerful factor in claim success is the Provider Specialty. This creates a 5.1% performance gap between the top (Orthopedics at 36.4% approval) and the bottom (Pediatrics at 31.3%). The Pediatrics specialty is the dataset's single largest bottleneck, not only for its low approval rate but also for holding the highest value of "Pending" claims at $1.74 million

2. **Patient Segmentation & Key Predictors(hive)**

This theme combines all patient-facing insights to build a clear picture of *which* demographic factors matter and, just as importantly, *which do not*.

Finding: Not all patient data is predictive. Factors like PatientGender, MaritalStatus, and Income have less than 2% variance and are negligible. The factors that do matter are PatientEmploymentStatus (where "Unemployed" has the highest approval at 36.3%, while "Student" and "Retired" are lowest at 32.6%) and PatientAge (where the "19-35" group is highest at 36.8%, and "65+" is lowest at 31.7%)**.**

3. **Targeted Financial Risk(hive)**

Finding: There is a major financial risk hidden within high-value claims. The top 5% of claims (those valued over $9510.27) are not treated with extra care; instead, they face an unusually high 36% denial rate.

   -

4. **Critical Financial Liability(MapReduce)**

Finding: The single most critical issue is the **$7.21 million (32.0%)** in financial exposure locked in 1,466 "Pending" claims. This backlog (32.6% of all claims) is more than double the standard industry benchmark of 15%. The risk strategy was defined by calculating both backlog rate by value and count.

5. **Operational & Temporal Inefficiency(PySpark + python hybrid)**

This theme groups all insights related to *how* and *when* a claim is processed, revealing significant inconsistencies.

Finding: The data shows a clear process lottery. Phone submissions (34.6%) are more effective than Online submissions (32.9%). Claims processed on a Tuesday (36.1%) are approved far more often than those on a Wednesday (32.0%). Finally, there is a predictable seasonal dip in Q2 (32.1% approval) compared to Q1 (34.6%).

---

6. Future Work

- Provider Performance Analysis – Extend provider-level insights using additional features like experience, location, and claim volume to identify performance gaps.
- High-Value Claim Monitoring – Implement scalable pipelines to flag and review high-value claims, reducing financial exposure.
- Pending Claims Prioritization – Use real-time big data workflows to track and prioritize pending claims, minimizing backlog.
- Operational & Temporal Optimization – Analyze submission channels, processing days, and seasonal trends to improve efficiency and approval consistency.
- Longitudinal Studies – Conduct long-term studies to monitor improvements in claim outcomes and operational efficiency post-intervention.
- Integration of External Datasets – Incorporate external data such as insurance history or provider certifications to enrich analysis and support predictive insights.

---

7. References

1. Course materials on Hive, MapReduce, and PySpark
2. Industry benchmark reports for claims processing:

Kodiak Solutions. (2024, May 21). *Rate of initial denials of medical insurance claims continued to rise in 2024*. Business Wire.

3. Hortonworks. (2018). *HDP sandbox user guide & HDFS documentation*. Cloudera. https://docs.cloudera.com/documentation/